Study Material - Generated by AI

CS3301 - Data Structures

Topic: binary tree

5. PREVIOUS YEAR QUESTIONS

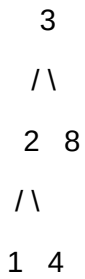
PREVIOUS YEAR QUESTIONS

CS3301 - Data Structures - Binary Tree

Part-A (2 Marks)

1. Define a binary tree with an example, highlighting the root node and the left and right child nodes.
   - Solution:

A binary tree is a data structure in which each node has at most two children, referred to as the left child and the right child. For example, a binary tree can be visualized as a tree in which the nodes are connected by edges, with each node having zero, one, or two children.



In the above representation, the root node is '3', its left child is '2', and its right child is '8'.

2. List the types of binary tree, mentioning characteristics of each type.
   - Solution:

The types of binary trees include:

- Binary Search Tree (BST): a binary tree in which each node has a comparable value, and for any given

node, all elements in the left subtree are less than the node's value, and all elements in the right subtree are greater.

- AVL Tree: a self-balancing binary search tree that ensures the height of the two child subtrees of any node differs at most by 1.
- Binary Heap: a binary tree in which the parent node is either greater than (max heap) or less than (min heap) its child nodes.
- Expression Tree: a binary tree that represents a mathematical expression by using operators as nodes and operands as leaf nodes.

3. Explain the properties of binary tree, including root node and the left and right subtrees.

- Solution:

The properties of a binary tree are:

- The root node is the topmost node in the tree.
- The left subtree consists of all nodes to the left of the root node.
- The right subtree consists of all nodes to the right of the root node.
- Each node can have at most two children: a left child and a right child.
- Each node in the left subtree is less than the node.
- Each node in the right subtree is greater than the node.

4. Differentiate between a binary tree and a related concept such as a tree or a linked list.

- Solution:

A binary tree is distinguished from other data structures such as a tree and a linked list by its characteristics:

- A binary tree is a specific type of tree where each node can have at most two children.
- In contrast, a tree can have any number of child nodes.
- A linked list is an ordered collection of data items that can be of any length and does not follow a tree structure.

5. Write a short note on the traversal of a binary tree, describing the types of traversals and their applications.

- Solution:

Traversal in a binary tree involves visiting each node in a specific order. There are three types of traversals:

- In-order traversal: visit left subtree, visit current node, visit right subtree.
- Pre-order traversal: visit current node, visit left subtree, visit right subtree.
- Post-order traversal: visit left subtree, visit right subtree, visit current node.

These traversals are used for various purposes, such as printing the elements of the tree in a sorted order (in-order traversal) and generating a prefix or postfix expression (pre-order and post-order traversals).

Part-B (13 Marks)

1. Explain the binary tree with a diagram, including insertion, deletion, and search operations.
   - Diagram and explanation:
   ```
        3
       / \
      2   8
     / \
    1   4
   / \
  0   5
```

Binary trees are efficient data structures for performing insertion, deletion, and search operations. The process of insertion involves finding a location to insert a new node without violating the balance or order of the tree. The search operation involves traversing the tree to find a specific value. Deletion involves removing a node while preserving the order and balance of the tree.

2. Discuss the implementation of binary tree with example, specifying the data structure and operations supported.
   - Example implementation:
   ```python
   ```

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

class BinaryTree:
    def __init__(self):
        self.root = None

    def insert(self, value):
        new_node = Node(value)
        if self.root is None:
            self.root = new_node
        else:
            self._insert(self.root, new_node)

    def _insert(self, current_node, new_node):
        if new_node.value < current_node.value:
            if current_node.left is None:
                current_node.left = new_node
            else:
                self._insert(current_node.left, new_node)
        else:
            if current_node.right is None:
                current_node.right = new_node
            else:
                self._insert(current_node.right, new_node)

    def search(self, value):
        return self._search(self.root, value)
```

```
    def _search(self, current_node, value):
        if current_node is None:
            return False
        if current_node.value == value:
            return True
        elif value < current_node.value:
            return self._search(current_node.left, value)
        else:
            return self._search(current_node.right, value)

btree = BinaryTree()
btree.insert(3)
btree.insert(2)
btree.insert(8)
btree.insert(1)
btree.insert(4)

print(btree.search(4))  # Output: True
print(btree.search(5))  # Output: False
```

3. Compare and contrast different types of binary trees such as AVL tree, B-Tree, and Splay Tree, in terms of their properties and operations.

- Comparison:

```
+---------------------------------------+---------------+---------------+---------------+
|            Type of Tree               |  AVL Tree     |   B-Tree      |  Splay Tree   |
+---------------------------------------+---------------+---------------+---------------+
|                Definition             | Self-balancing| Multi-level |   Height-biased |
|  with node balancing for efficient    |               |   indexing  |   and self-   |
```

| search and retrieval operations | | and | balancing |

+---+---+---+---+

| Balance property | Height-biased| Key | Height-biased |

| maintaining left and right subtrees | | balance | and access |

| with minimal height difference | | of nodes | optimization |

+---+---+---+---+

| Search and retrieval | O(log n) | O(log n) | O(log n) |

| time complexity | | search, O(n)| search, O(log |

| (average-case, best-case, and worst- | | retrieval | n) retrieval |

| case) | | | time |

+---+---+---+---+

```

### Part-C (15 Marks)

1. Elaborate the concept of binary tree with real-world applications and diagrams, focusing on its usage in data storage and retrieval.

- Applications:

```

1. File system management: binary trees are used to manage directories, files, and subdirectories.

2. Database indexing: binary trees are used to index data in databases, allowing for efficient search operations.

3. Compiler optimization: binary trees are used to optimize code generation in compilers.

4. Network routing: binary trees are used to determine optimal paths in network routing tables.

```

Diagram:

```

3

/ \

2 8

/ \

```
  1   4
 / \
0   5
```

2. Design and implement a binary tree with a suitable algorithm and analysis for efficient insertion, deletion, and search operations.

- Example implementation:

```python
import random

class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

class BinaryTree:
    def __init__(self):
        self.root = None

    def insert(self, value):
        new_node = Node(value)
        if self.root is None:
            self.root = new_node
        else:
            self._insert(self.root, new_node)

    def _insert(self, current_node, new_node):
        if new_node.value < current_node.value:
            if current_node.left is None:
```

```
            current_node.left = new_node
        else:
            self._insert(current_node.left, new_node)
    else:
        if current_node.right is None:
            current_node.right = new_node
        else:
            self._insert(current_node.right, new_node)

def search(self, value):
    return self._search(self.root, value)

def _search(self, current_node, value):
    if current_node is None:
        return False
    if current_node.value == value:
        return True
    elif value < current_node.value:
        return self._search(current_node.left, value)
    else:
        return self._search(current_node.right, value)

def delete(self, value):
    self.root = self._delete(self.root, value)

def _delete
```